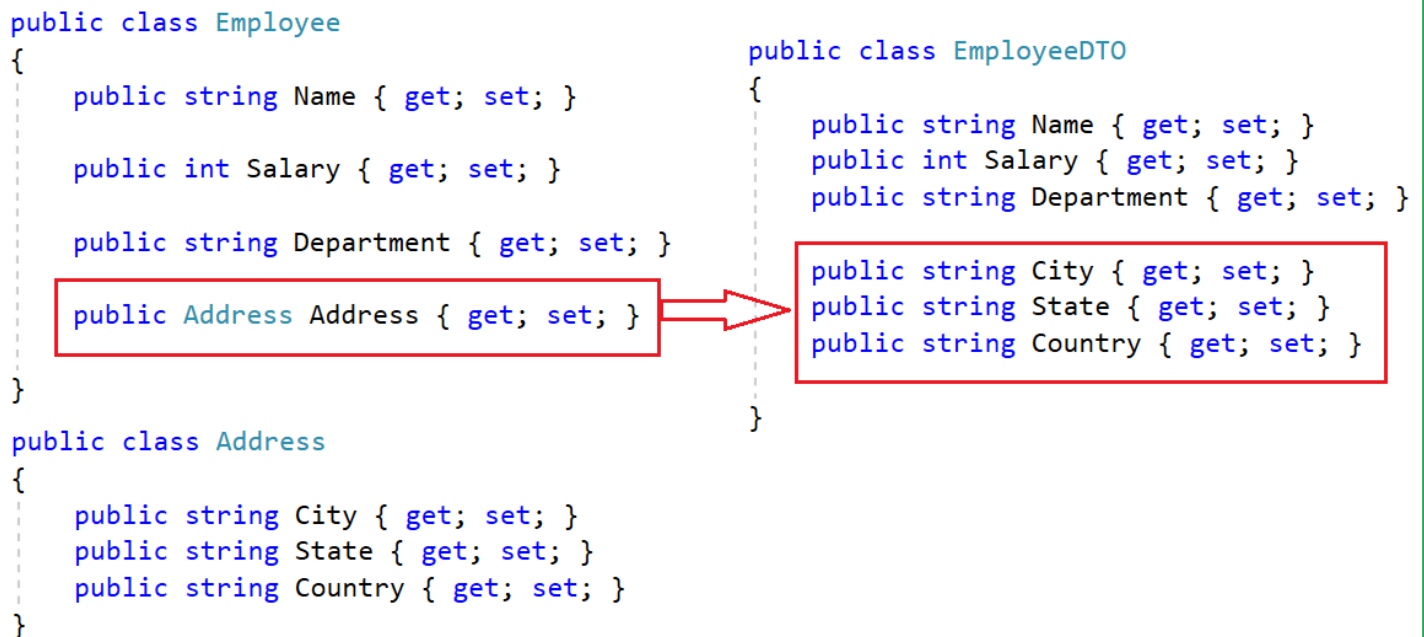


How to Map Complex Type to Primitive Type using AutoMapper in C#

In this article, I am going to discuss **How to Map Complex Type to Primitive Type using AutoMapper in C#** with Examples. Please read our previous article where we discussed the [Automapper Complex Mapping in C#](#) with Examples. At the end of this article, you will understand when and how to map complex types to primitive types using AutoMapper.

When to Map Complex Type to Primitive Type using AutoMapper in C#?

When one class contains primitive types or you can say the simple types and the other class contains complex types involved in the mapping then in such scenarios we need to Map the Complex Type to the Primitive Types. Let us understand how to map Complex type to Primitive Type using AutoMapper in C# with an example. Here in this demo, we are going to use the following three classes (Employee, EmployeeDTO and Address).



```
public class Employee
{
    public string Name { get; set; }

    public int Salary { get; set; }

    public string Department { get; set; }

    public Address Address { get; set; }
}

public class Address
{
    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
}

public class EmployeeDTO
{
    public string Name { get; set; }
    public int Salary { get; set; }
    public string Department { get; set; }

    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
}
```

As shown in the above image, here, we want to Map the complex property **Address** of the Employee class to the **City, State, and Country** properties of **EmployeeDTO** class. So, first, create a class file with the name Address.cs and then copy and paste the following code into it.

```
namespace AutoMapperDemo
{
    public class Address
    {
        public string City { get; set; }
        public string State { get; set; }
        public string Country { get; set; }
    }
}
```

Next, create a class file with the name Employee.cs and then copy and paste the following code into it.

```
namespace AutoMapperDemo
{
    public class Employee
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public Address Address { get; set; }
    }
}
```

Finally, create a class file with the name EmployeeDTO.cs and then copy and paste the following code into it.

```
namespace AutoMapperDemo
{
    public class EmployeeDTO
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public string City { get; set; }
        public string State { get; set; }
        public string Country { get; set; }
    }
}
```

Mapping Complex type to Primitive Type using AutoMapper in C#

In order to map the Complex Type to the Primitive Types, we need to use the ForMember method of AutoMapper and we also need to specify the source and target properties. Here, we need to map the City, State, and Country properties of the Address object to the City, State, and Country properties of EmployeeDTO class. So, create a class file with the name **MapperConfig.cs** and then copy and paste the following code into it. Here, we are creating one static method called **InitializeAutomapper** and inside this method, we are writing all the mapping code. As you can see, here, we are mapping the City, State, and Country properties of the Address object to the City, State, and Country properties of the EmployeeDTO object.

```
using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
            var config = new MapperConfiguration(cfg => {
                //Configuring Employee and EmployeeDTO
                cfg.CreateMap<Employee, EmployeeDTO>()
            })
        }
    }
}
```

```

        //Provide Mapping Information for City Property
        .ForMember(dest => dest.City, act => act.MapFrom(src => src.Address.City))
        //Provide Mapping Information for State Property
        .ForMember(dest => dest.State, act => act.MapFrom(src =>
src.Address.State))
        //Provide Mapping Information for Country Property
        .ForMember(dest => dest.Country, act => act.MapFrom(src =>
src.Address.Country));
    });

    //Create an Instance of Mapper Class and return that Instance
    var mapper = new Mapper(config);
    return mapper;
}
}
}

```

As shown in the above code. we mapped each property from the complex type (**Address**) of the source object (**Employee**) to the correspondent properties of the destination object (**EmployeeDTO**). Next, modify the Main method of the Program class as follows.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Creating an Instance of Address Entity
            Address empAddress = new Address()
            {
                City = "Mumbai",
                State = "Maharashtra",
                Country = "India"
            };
            //Creating and Instance of Employee Entity
            Employee emp = new Employee
            {
                Name = "James",
                Salary = 20000,
                Department = "IT",
                Address = empAddress
            };
            //Initialize the AutoMapper
            var mapper = MapperConfig.InitializeAutomapper();
            //Way1: Mapping emp Object with EmployeeDTO Object
            var empDTO = mapper.Map<EmployeeDTO>(emp);
            //Way2: Mapping emp Object with EmployeeDTO Object
            //var empDTO = mapper.Map<Employee, EmployeeDTO>(emp);
            Console.WriteLine("Name:" + empDTO.Name + ", Salary:" + empDTO.Sal
Department:" + empDTO.Department);

```

```

        Console.WriteLine("City:" + empDTO.City + ", State:" + empDTO.State + ", Country:" + empDTO.Country);
        Console.ReadLine();
    }
}

```

Now run the application and you will see the output as expected as shown in the below image.

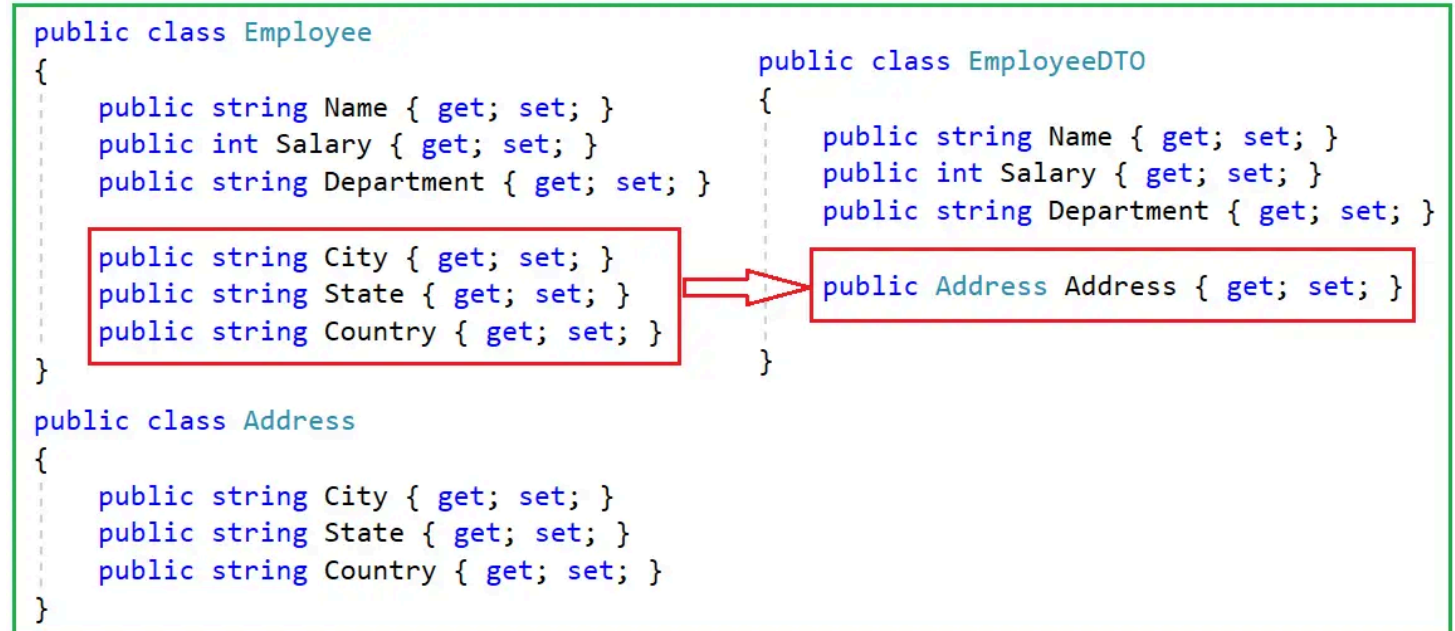
```

Name:James, Salary:20000, Department:IT
City:Mumbai, State:Maharashtra, Country:India

```

Mapping Primitive Properties to a Complex Type using AutoMapper in C#:

Now we want to map the primitive type properties of the source object to a complex type of the destination object as shown in the image below.



As you can see in the above image, now we want to map the **City, State, and Country** Properties of the source (**Employee**) object to the **Address** property of the destination (**EmployeeDTO**) object. So, let us first modify the **Employee** class as follows to include the primitive properties by removing the complex address property.

```

namespace AutoMapperDemo
{
    public class Employee
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public string City { get; set; }
        public string State { get; set; }
    }
}

```

```

        public string Country { get; set; }
    }
}

```

Next, modify the EmployeeDTO class as follows to include a complex property instead of primitive properties to store the address.

```

namespace AutoMapperDemo
{
    public class EmployeeDTO
    {
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public Address Address { get; set; }
    }
}

```

Now, we need to Map the Primitive Properties of the Source Object to a Complex Property of the Destination Object. To do so, modify the MapperConfig class as follows.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            var config = new MapperConfiguration(cfg => {
                cfg.CreateMap<Employee, EmployeeDTO>()
                    .ForMember(dest => dest.Address, act => act.MapFrom(src => new
                    {
                        City = src.City,
                        State = src.State,
                        Country = src.Country
                    }));
            });
            //Create an Instance of Mapper Class and return that Instance
            var mapper = new Mapper(config);
            return mapper;
        }
    }
}

```

As you can see in the above code, we project an object (**Address type**) using the **MapForm** option and City, State, and Country Values are coming from the Source object. Next, modify the Main method of the Program class as follows.

```

using System;
namespace AutoMapperDemo
{

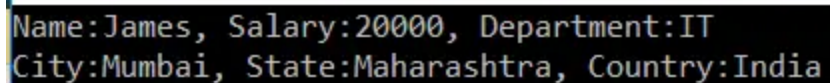
```

```

class Program
{
    static void Main(string[] args)
    {
        //Creating an Instance of Employee Entity
        Employee emp = new Employee
        {
            Name = "James",
            Salary = 20000,
            Department = "IT",
            City = "Mumbai",
            State = "Maharashtra",
            Country = "India"
        };
        //Initialize the AutoMapper
        var mapper = MapperConfig.InitializeAutomapper();
        //Way1: Mapping emp Object with EmployeeDTO Object
        var empDTO = mapper.Map<EmployeeDTO>(emp);
        //Way2: Mapping emp Object with EmployeeDTO Object
        //var empDTO = mapper.Map<Employee, EmployeeDTO>(emp);
        Console.WriteLine("Name:" + empDTO.Name + ", Salary:" + empDTO.Sal
Department:" + empDTO.Department);
        Console.WriteLine("City:" + empDTO.Address.City + ", State:" +
empDTO.Address.State + ", Country:" + empDTO.Address.Country);
        Console.ReadLine();
    }
}

```

When you run the application, it will display the output as expected as shown in the below image.



```

Name:James, Salary:20000, Department:IT
City:Mumbai, State:Maharashtra, Country:India

```

In the next article, I am going to discuss [Reverse Mapping using AutoMapper in C#](#) with examples. Here, in this article, I try to explain **how to map Complex type to Primitive Type using AutoMapper in C# and Vice Versa** with examples. I hope this Complex type to Primitive Type using AutoMapper in the C# article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.